

Design and Analysis of Algorithms a.k.a. Analysis of Algorithms

Special Section 2024 (Summer 24)

Quiz / Assignment 02 - 17-07-2024 Due on: 22-07-24

1. Prove the correctness of the following algorithm to return about the presence of x in array A.

LinearSearch(A, x)

```
1. found = false
2. for j = 1 to A.length
3.     if x = A[j]
4.         found = true
5. return found
```

2. Analyze the above **LinearSearch** algorithm for the running time, clearly specify its input size.
3. Analyze following **SelectionSort** algorithm for correctness and running time.

SelectionSort(A)

```
1. for j = 1 to A.length
2.     mj = j // index of minimum in subarray A[j..n]
3.     for k = j to A.length
4.         if A[k] < A[mj]
5.             mj = k
6.     swap A[j] with A[mj]
```

4. Analyze the following **MatrixSum** algorithm for addition of two matrices, assumed compatible for addition. The running time should be function of two variables: rows and columns of matrices.

MatrixSum(A, B)

```
1. p = A.rows
2. q = A.cols
3. let C be a new p x q matrix
4. for j = 1 to p
5.     for k = 1 to q
6.         C[j,k] = A[j,k] + B[j,k]
7. return C
```

5. Analyze the following **SqMatrixSum** algorithm for addition of two square matrices. The running time should be function of one variable: rows or columns of matrices.

SqMatrixSum(A, B)

```
1. n = A.rows
2. let C be a new n x n matrix
3. for j = 1 to n
4.     for k = 1 to n
5.         C[j,k] = A[j,k] + B[j,k]
6. return C
```

Design and Analysis of Algorithms a.k.a. Analysis of Algorithms

Special Section 2024 (Summer 24)

Quiz / Assignment 02 - 17-07-2024 Due on: 22-07-24

6. Which type of arrays, the following sorting algorithm, can sort? Explain your answer.

```
MERRTION-SORT(A)
1  for j = 2 to A.length
2      key = A[j]
3      // Insert A[j] into the sorted sequence A[1 .. j - 1]
4      i = j - 1
5      while i > 0
6          A[i + 1] = A[i]
7          i = i - 1
8      A[i + 1] = key
```

7. Following is a snapshot from the book, describe its context comprehensively.

$$\frac{2 \cdot (10^7)^2 \text{ instructions}}{10^{10} \text{ instructions/second}} = 20,000 \text{ seconds (more than 5.5 hours)}$$

8. Obtain the recurrence for the following algorithm that processes array A from a lower index s to a higher index e.

SillyCode(A, s, e)

1. if s = e
2. print((A[s]+A[e])/2)
3. SillyCode(A, s+1, e)
4. print(A[s])
5. SillyCode(A, s, e-1)
6. print(A[e])

9. Write straight forward recursive functions for computation of 1) factorial of n, and 2) nth Fibonacci number. Obtain their recurrences and solve them using recursion tree method.
10. Focus on the following code and its recurrence and try to understand it, despite your feelings. Finally, write down your observations and concerns.

```
RECURSIVE-MATRIX-CHAIN(p, i, j)
1  if i == j
2      return 0
3  m[i, j] = ∞
4  for k = i to j - 1
5      q = RECURSIVE-MATRIX-CHAIN(p, i, k)
          + RECURSIVE-MATRIX-CHAIN(p, k + 1, j)
          + pi-1pkpj
6      if q < m[i, j]
7          m[i, j] = q
8  return m[i, j]
```

$$T(1) = 1,$$
$$T(n) = 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) \quad \text{for } n > 1.$$

Noting that for $i = 1, 2, \dots, n-1$, each term $T(i)$ appears once as $T(k)$ and once as $T(n-k)$, and collecting the $n-1$ 1s in the summation together with the 1 out front, we can rewrite the recurrence as

$$T(n) = 2 \sum_{i=1}^{n-1} T(i) + n.$$

----- The end -----

Design and Analysis of Algorithms a.k.a. Analysis of Algorithms
Special Section 2024 (Summer 24)

Quiz / Assignment 02 - 17-07-2024 Due on: 22-07-24

For better readability here are magnified versions of the snapshots in last question.

RECURSIVE-MATRIX-CHAIN(p, i, j)

```
1  if  $i == j$ 
2      return 0
3   $m[i, j] = \infty$ 
4  for  $k = i$  to  $j - 1$ 
5       $q = \text{RECURSIVE-MATRIX-CHAIN}(p, i, k)$ 
            $+ \text{RECURSIVE-MATRIX-CHAIN}(p, k + 1, j)$ 
            $+ p_{i-1} p_k p_j$ 
6      if  $q < m[i, j]$ 
7           $m[i, j] = q$ 
8  return  $m[i, j]$ 
```

$$T(1) = 1,$$

$$T(n) = 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) \quad \text{for } n > 1.$$

Noting that for $i = 1, 2, \dots, n-1$, each term $T(i)$ appears once as $T(k)$ and once as $T(n-k)$, and collecting the $n-1$ 1s in the summation together with the 1 out front, we can rewrite the recurrence as

$$T(n) = 2 \sum_{i=1}^{n-1} T(i) + n.$$